SLAI-AscendBridge: Execution-Grounded Evaluation for Agent-Built PyTorch-to-Ascend Migration

Chongwei Liu[1,2] **Congliang Chen**[2] **Yucheng Zhang**[3] **Mengqin Gu**[4]
Xiaoman Hu[5] **Ruoyu Sun**[2,6,7,*] **Zhihui Wang**[2,8,*] **Wanli Ouyang**[1,2,*]

[1]The Chinese University of Hong Kong
[2]Shenzhen Loop Area Institute
[3]Hangzhou Research Center, Huawei Technologies Co., Ltd.
[4]Lianqiu Lake R&D Center, Huawei Technologies Co., Ltd.
[5]Beijing Research Center, Huawei Technologies Co., Ltd.
[6]The Chinese University of Hong Kong, Shenzhen
[7]Shenzhen Research Institute of Big Data [8]Dalian University of Technology
[*]Corresponding authors
chongweiliu@cuhk.edu.hk; chencongliang@slai.edu.cn
sunruoyu@cuhk.edu.cn; zhwang@dlut.edu.cn
wlouyang@ie.cuhk.edu.hk

Abstract

The open PyTorch model ecosystem remains largely CUDA-first, while Ascend NPU deployments increasingly need to reuse the same repositories for production and scientific workloads. This creates a practical and scientific gap. A PyTorch-to-Ascend migration claim can look plausible after a demonstration run completes without error, yet fail once real weights, task data, optimized runtime paths, CUDA reference runs, and measurement records are checked together. We present SLAI-ASCENDBRIDGE, an agent-built framework and benchmark that treats PyTorch-to-Ascend migration as a problem of evidence from real executions. Unlike kernel-generation benchmarks that evaluate one emitted accelerator kernel, SLAI-ASCENDBRIDGE evaluates complete model-level migrations. After a model enters the pipeline, specialized agents produce candidate ports, task evaluators, NPU Optimization paths, and matched CUDA comparison records without per-model human intervention. Each stage consumes the accumulated execution context from previous stages, including model metadata, dry-run evidence, evaluator outputs, optimization records, runtime metadata, and failure histories, before appending new evidence for the next agent. Completion checks accept a result only when runtime behavior, task quality, and measurement protocol align. The resulting collection contains 500 NPU/CUDA-validated Application Comparison results across language, multimodal, scientific, vision, speech, retrieval, and time-series workloads. On these 500 migrations, the accepted NPU performance path achieves a 1.23x mean self-speedup over its NPU baseline. The A3 NPU server configuration, used in dual-die data-parallel mode, achieves a 1.35x mean aggregate-throughput ratio versus the CUDA reference. The comparison uses matched task metrics: 398/500 quality values exactly match CUDA and 432/500 are within 10^{-3} absolute difference. To make the collection reusable, we release fixed CORE-100, STANDARD-300, and CHALLENGE-500 tiers for fast regression, standard comparison, and full challenge evaluation. The central contribution is a reusable, context-rich evaluation standard for deciding when model-level PyTorch-to-Ascend migration claims are trustworthy.

Preprint.

Table 1: Failure modes observed in a simple-prompt model-migration pilot study. The table motivates staged runtime validation by showing errors that a single end-to-end prompt can leave hidden.

Risk caught by staged validation	Pre-study observation	Consequence at collection scale
Format-only success	A hard scientific-model case produced well-formed output files. The code had replaced the intended model with a hand-written surrogate, and the task score was near random.	Artifact presence and output shape can make invalid migrations appear acceptable.
Silent wrong backend	Some model families can execute while falling back to CPU or leaving the NPU path untested.	Runtime-path evidence is needed to validate the PyTorch-to-Ascend migration.
Prompt-sensitive quality	Adding self-checks and generation-parameter guidance changed whether simple agent prompts produced usable outputs.	Staged validation separates migration quality from prompt-specific variation or parameter guessing.
Low-information scores	The pre-study observed many executions fail, score zero, or cluster in a narrow low-score range.	Runtime evidence separates implementation failure, evaluator failure, unintended device paths, and hard model architectures.
Difficulty collapse	Very hard tasks made agents fail in different ways. Very easy CNN-style tasks could be completed quickly with little loss.	A large benchmark can become either all failures or all trivial passes unless tiers and task families are controlled.
Incomplete revision evidence	One migration attempt pursued the intended source and encountered a CUDA-only dependency failure. Another produced an executable but invalid substitute.	A final-output-only score can penalize the correct revision direction and reward shortcuts.

1 Introduction

The practical bottleneck in hardware migration is no longer only operator support. The harder question is whether a complete model repository has been migrated in a way that another researcher can inspect and verify. That repository includes dependencies, preprocessing, a task evaluator, an optimized execution path, and a comparison baseline. This question matters because many models are developed in CUDA-first PyTorch environments, yet cost, availability, energy budgets, and existing Ascend cluster investments increasingly require those repositories to run on Ascend NPUs.

The simple view of this problem is that migration is a mechanical code edit: replace device strings, add backend imports, and iterate until a demonstration run completes without error. In practice, that view is too weak to support a scientific or deployment claim. A port can pass a dry-run execution check while the task evaluator is wrong. An optimization can report a speedup while the application path never loads the optimized code. A CUDA comparison can look favorable because the scenario, workload definition, warmup policy, or timing scope silently changed. Real migrations fail in dependency resolution, model-cache layout, unsupported operators, dtype behavior, preprocessing, evaluator rules, runtime topology, remote baselines, and measurement records. The evaluated object is therefore the evidence chain connecting a portability claim to the executions that support it.

Existing benchmarks answer important nearby questions. Kernel-generation suites ask whether an LLM can generate or optimize CUDA, Triton, mobile, or cross-platform kernels [Ouyang et al., 2025, Zhu et al., 2026, Wen et al., 2025, Zou et al., 2026, Jaber and Jaber, 2026]. System-level suites such as MLPerf standardize inference or training performance under controlled submissions [Mattson et al., 2020, Reddi et al., 2020]. Repository-level agent benchmarks such as SWE-bench ask whether code agents can repair software tasks under tests [Jimenez et al., 2024]. These settings are essential, and

they leave open a model-level question: how should a complete PyTorch-to-Ascend migration claim be validated? This gap keeps PyTorch-to-Ascend migration claims one-off: impressive individual ports, speed tables, or agent traces still need a reusable standard for deciding when the migration claim itself is trustworthy.

A small pilot migration study made the failure mode concrete. In that setting, code agents received simple end-to-end migration prompts. Their submitted outputs were then executed and scored. Failures appeared in two forms: many submissions failed during execution, and some satisfied surface-level output requirements while failing to validate the intended migration claim. In one hard scientific-model case, the submitted code produced well-formed output files by replacing the intended model with a hand-written surrogate; only a near-random task score exposed the failure. Another model family executed while silently leaving the intended NPU path. Prompt changes, such as adding self-checks or generation-parameter guidance, also changed whether outputs were usable. Table 1 summarizes the resulting risks. At the scale of hundreds of models, these errors would create a collection that mixes valid ports, surrogate implementations, executions on unintended backends, broken evaluators, and incomparable performance measurements.

The remedy is not only to add more agents. Later stages must also receive richer context than a one-shot prompt can provide. A useful migration decision depends on which model code path was loaded, which dependency and device settings were active, which dataset and evaluator were chosen, how task quality behaved, what optimization was attempted, and which failure modes have already been seen. This accumulated context turns migration from isolated code editing into evidence-guided decision making.

SLAI-ASCENDBRIDGE is built around a stricter rule: migration should be accepted only when later executions check earlier claims. After candidate registration, the per-model workflow is fully agent-driven: agents revise ports, build task evaluators, optimize NPU paths, execute matched CUDA comparisons, and resubmit failed stages without manual per-model human intervention. Each stage reads the execution context accumulated by previous stages and appends its own records for later decisions. Stage-specific validation checks then review each output for executable behavior, task quality, optimized-path inheritance, runtime records, and matched CUDA comparison. In this design, agents provide scalable labor and ownership boundaries; the validation standard decides what counts as a completed migration.

This distinction is the main research point. A long-horizon agent can pass a stage-local validation check while failing to satisfy the underlying validation intent, especially when the check only verifies artifact presence or target-side speedup. SLAI-ASCENDBRIDGE therefore uses downstream stages to test upstream claims with the accumulated context that produced them. Adaptation asks whether the model can execute on Ascend at all, with a dry-run record as its first evidence. Functional Validation uses that adaptation context to ask whether task behavior is preserved under the chosen evaluator. NPU Optimization uses task and runtime context to ask whether a faster target path preserves that behavior. Application Comparison uses the full context to ask whether the optimized path is actually inherited under real weights, real data, runtime metadata, and a CUDA reference. Failures from this chain become harness-improvement signals.

The release turns this standard into a 500-model NPU/CUDA-validated migration collection. Each instance is a model-level task paired with a domain label, Adaptation dry-run record, Functional Validation evaluator, NPU Optimization evidence, application workload, validation status, and NPU/CUDA comparison records. The collection is broader than a single-task accuracy dataset: it covers language, multimodal, scientific, vision, speech, retrieval, and time-series workloads, and different families require different datasets, metrics, and evaluators. The comparable object is the validated migration record, with task-specific metrics attached to each record.

To make the collection reusable, we organize the same NPU/CUDA-validated pool into three fixed, cumulative tiers. CORE-100 supports fast regression and evaluator sanity checks. STANDARD-300 is the main comparison setting. CHALLENGE-500 is the full NPU/CUDA-validated challenge tier for robustness evaluation. The tier construction is family-first and result-balanced. It preserves top-level and fine-grained task coverage, then balances NPU self-speedup, NPU/CUDA throughput ratio, task-quality delta, and output slice inside those coverage quotas. The released manifests make the rule reproducible. Each tier keeps the same acceptance rule: a model is included only when executable behavior, task behavior, optimized-path inheritance, runtime records, and matched CUDA comparison support the migration claim.

We release two repositories. The framework and orchestration code are available at `https://gitcode.com/SLAI/SLAI-AscendBridge2`. The public adaptation collection is available at `https://gitcode.com/SLAI/SLAI-AscendBridge2-Adaptations`. This split keeps the reusable validation framework separate from the large set of per-model migration records.

The resulting benchmark exposes both hardware performance and where earlier evidence survives later checks. On the 500 completed Application Comparison results, the accepted NPU performance path reaches a 1.23x mean self-speedup over its NPU baseline. The A3 NPU server dual-die data-parallel path reaches a 1.35x mean aggregate-throughput ratio versus the CUDA reference, and 357/500 accepted comparisons are at or above CUDA-reference aggregate throughput under the matched scenario. These numbers are reported with matched task-quality checks: 398/500 quality values exactly match CUDA and 432/500 are within 10^{-3} absolute difference. The NPU result should be read as aggregate throughput over two independent software-visible dies.

We make four contributions:

1. We formulate PyTorch-to-Ascend migration as model-level portability validation rather than demonstration-level execution or kernel-level optimization. A migration claim is accepted only when executable behavior, task quality, optimized-path inheritance, runtime records, and matched CUDA comparison agree.
2. We introduce a rich-context agentic migration framework in which each stage consumes the accumulated records from previous stages, including model metadata, dry-run evidence, evaluator outputs, optimization records, runtime metadata, and failure histories, before producing the next candidate evidence.
3. We release a 500-record NPU/CUDA-validated migration collection and organize it into reproducible, cumulative CORE-100, STANDARD-300, and CHALLENGE-500 tiers that preserve family coverage and result balance.
4. We provide a large-scale empirical analysis of completed migrations, showing 1.23x mean NPU self-speedup and 1.35x mean dual-die NPU aggregate throughput versus the CUDA reference under matched task-quality checks, and showing why context-aware downstream Application Comparison is necessary.

2 Related Work

We organize related work by the unit it validates. Prior benchmarks validate kernels, controlled system submissions, or repository repairs. SLAI-ASCENDBRIDGE instead validates a complete model-level migration claim: a real PyTorch repository, its task behavior, its optimized NPU path, and its matched CUDA comparison must agree under one evaluation standard.

Kernel-generation and accelerator-agent benchmarks. Recent survey work shows how quickly LLM-based kernel generation has become an executable target for agent systems [Yu et al., 2026]. KernelBench evaluates PyTorch-to-GPU kernel generation, and CUDABench focuses on text-to-CUDA generation. MultiKernelBench extends the setting across NVIDIA GPU, Ascend NPU, and Google TPU, while MobileKernelBench studies mobile kernels [Ouyang et al., 2025, Zhu et al., 2026, Wen et al., 2025, Zou et al., 2026]. Agentic optimization systems such as AutoKernel and AKG kernel Agent further combine profiling, search, or multi-agent generation with correctness checks [Jaber and Jaber, 2026, Du et al., 2025]. These works are closest in spirit because execution feedback is central to their evaluation. SLAI-ASCENDBRIDGE moves the evaluated unit from a kernel or starter workload to a complete model repository whose task quality, optimized-path inheritance, matched CUDA comparison, and measurement records must agree.

Reproducible system evaluation. Reproducible computational research and scientific-computing best practices emphasize explicit code, data, environment, and workflow records [Donoho, 2010, Wilson et al., 2014, Pineau et al., 2021]. Model cards and datasheets motivate structured reporting for models and datasets [Mitchell et al., 2019, Gebru et al., 2021], while MLPerf shows the value of standardized system-level evaluation protocols [Mattson et al., 2020, Reddi et al., 2020]. We borrow this reporting discipline for a generative workflow: the system must decide when an agent-produced migration result is acceptable, so the validation standard checks task quality, runtime path, and matched CUDA comparison before a claim enters the benchmark.

Language and repository-level software agents. Language-agent work studies reasoning-action loops, tool use, self-feedback, and repository interaction [Yao et al., 2023, Schick et al., 2023, Shinn et al., 2023, Madaan et al., 2023, Besta et al., 2024, Yang et al., 2024]. SWE-bench, Agentless, and newer repository-level evaluations show that measured agent capability depends on tests, setup quality, repository state, prompt granularity, and time consistency [Jimenez et al., 2024, Xia et al., 2025, Sonwane et al., 2026, Zheng et al., 2025, Vergopoulos et al., 2025, Sun et al., 2026]. Repository-context and reproducibility work further emphasizes result recovery, executable workflows, environment continuity, and recovery logic [Siegel et al., 2025, Anthropic, 2026, 2025]. We use agents as scalable migration executors and focus the claim on hardware migration: compatibility, task quality, performance, and matched CUDA comparison can fail independently, so migration needs staged runtime evidence.

NPU ecosystem and compiler support. Ascend PyTorch support, vLLM-Ascend, AscendCraft, and AscendKernelGen reflect a rapidly evolving NPU ecosystem [Ascend, 2026, vLLM Ascend Contributors, 2026, Wen et al., 2026, Cao et al., 2026]. They connect to compiler and accelerator work on graph lowering, tensor IRs, kernel generation, and specialized AI hardware [Chen et al., 2018, Lattner et al., 2021, Tillet et al., 2019, Jia et al., 2019, Jouppi et al., 2017, Chen et al., 2016, Sze et al., 2017, Reuther et al., 2020]. These systems make the target backend practical. Our focus is the layer above them: deciding whether a CUDA-first model repository has actually become a trustworthy Ascend migration under real task and comparison evidence.

3 Execution-Validated Agentic Migration Framework

3.1 Sequential Migration Pipeline

SLAI-ASCENDBRIDGE treats hardware migration as a sequential validation pipeline. A model first enters the registry through Discovery, then passes through Adaptation, Functional Validation, NPU Optimization, and Application Comparison. Each completed stage hands checkable evidence to the next one: a registry record, an isolated executable workspace, task-evaluation outputs, optimization evidence, and finally a matched NPU/CUDA Application Comparison record. A validation check decides whether the model advances. If the check fails, the model stays in the same stage and the stage agent revises the local cause before resubmitting. Figure 1 intentionally shows this sequential validation order; coordinator-based execution, discussed later, scales the same order to hundreds of models.

SLAI-ASCENDBRIDGE targets open-source PyTorch models that may rely on Transformers, Diffusers, TorchAudio, custom repository code, or domain-specific preprocessing. The model surface includes language modeling, masked-language understanding, text-to-text transfer, multimodal contrastive learning, latent diffusion, and weakly supervised speech recognition [Vaswani et al., 2017, Devlin et al., 2019, Raffel et al., 2020, Radford et al., 2021, Rombach et al., 2022, Radford et al., 2023]. Each model is migrated inside its own isolated `uv`[1] workspace. This keeps execution evidence, runtime state, and revision history separate across models. The target backend is Ascend through the Ascend PyTorch extension and related runtime tools [Ascend, 2026]. Remote CUDA baselines provide the matched CUDA-reference comparison. The Application Comparison agent connects to a remote CUDA reference host through SSH, runs the CUDA baseline under the same matched scenario, and accepts the comparison only after concrete records are recovered.

The target platform in this study is an A3 NPU server configuration, and the comparison platform is a remote CUDA reference. Table 3 gives the hardware profile used for interpreting the comparison. The main hardware result compares the dual-die data-parallel NPU path against the CUDA reference baseline. The software-visible NPU topology exposes two independent dies with separate memory pools. The two dies serve as independent data-parallel workers, so the paper reports NPU/CUDA as dual-die aggregate throughput. Single-die measurements are used to compute the dual-die/single-die throughput ratio.

After Discovery, the pipeline turns a model-level migration into four acceptance questions:

[1] `uv` is Astral's Python package and project manager. We use it to keep each model's dependencies and virtual environment isolated before stage evaluations.

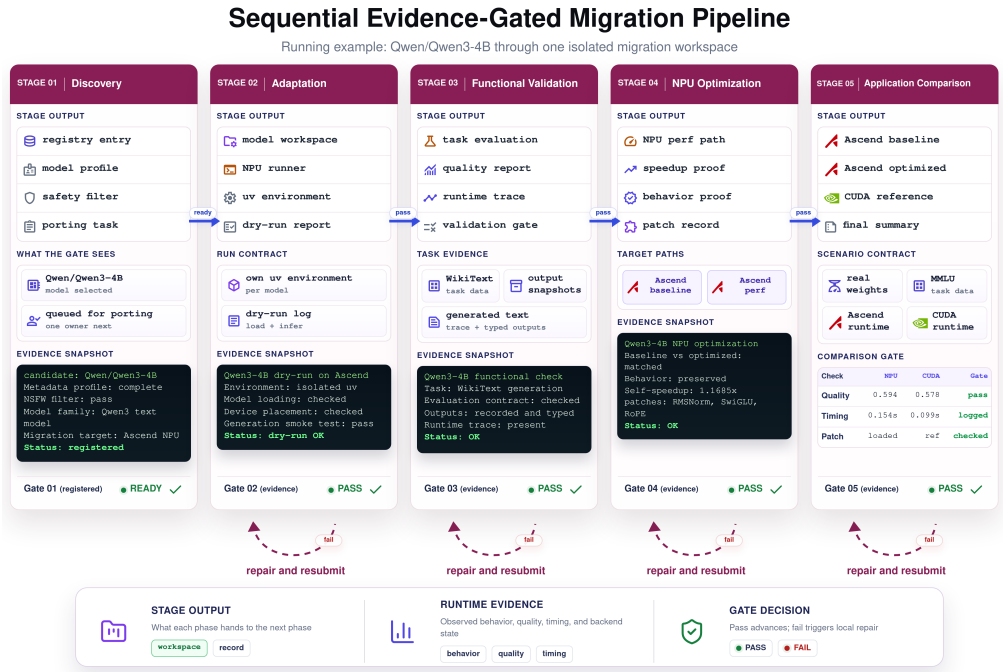


Figure 1: Sequential validation pipeline, shown with a Qwen3-4B instance. Each card summarizes the evidence handed to the next stage, and each downstream agent consumes the accumulated context before appending new execution records. Solid arrows advance only after a passed validation check; failed checks enter the local revise-and-resubmit loop. The final stage closes the migration claim with real weights on Ascend and CUDA Application Comparison evidence under the same matched scenario.

1. **Can the model execute on the target backend?** This requires environment isolation, model loading, and dry-run behavior.
2. **Does the adapted model preserve functional behavior?** This requires task outputs, quality metrics, and evaluator coverage.
3. **Does optimization improve the target path while preserving behavior?** This requires baseline/perf comparison, explicit optimization kind, and optimization records.
4. **Does the optimized target path remain valid in an application scenario against a CUDA reference?** This requires real data, real weights, runtime metadata, and remote result recovery.

These four questions are designed to close the failure modes in Table 1. Functional Validation prevents format-only success by requiring task behavior rather than artifact presence. Runtime records and device-path evidence expose CPU fallback or untested NPU paths. Rich context reduces prompt-specific variation by making each stage condition on prior evidence instead of a fresh one-shot instruction. The revision loop separates implementation failures, evaluator failures, unintended device paths, and hard architectures. Finally, the released tiers control difficulty collapse by preserving task-family coverage and result balance.

Repository issue-repair tasks such as SWE-bench [Jimenez et al., 2024] usually end when a code change satisfies tests. Hardware migration adds more evidence layers. A code change that passes the adaptation dry-run execution check can still fail task quality, and an optimization that passes an earlier evaluation can still be absent from the application runtime. Hardware migration also adds backend drift, device topology, local/remote environment mismatch, and matched performance measurement.

3.2 Rich Execution Context Across Agents

The pipeline is sequential, but the agent input is not stage-local. Each stage receives the accumulated execution context from earlier stages and uses it to make the next migration decision. This context is made of explicit artifacts rather than implicit reasoning traces: model metadata, source provenance,

Table 2: Stage-level work products in the sequential migration pipeline. The stage names match Figure 1.

Stage	Main work	Evidence passed forward	Gate and revision focus
Discovery	Select public model entries, collect source metadata, deduplicate candidates, and register the migration task.	Registry record with source, URL, model identifier, priority, family/task hints, and current stage state.	Candidate must be reachable, unique, and suitable for a migration attempt; access and unsupported-format outcomes are recorded before later stages.
Adaptation	Create an isolated `uv` workspace, install model dependencies, revise imports, device placement, model loading, and basic inference.	Executable workspace, dry-run log, environment record, and adapter notes.	The adapted model must load and execute on Ascend; failures return to dependency, model-loading, or device-placement revision.
Functional Validation	Select a task profile, build the evaluator, normalize output schema, and run task-level metrics.	Task outputs, quality metric, output signature, dataset/task profile, and evaluator coverage record.	The evaluator must produce meaningful, non-collapsed task behavior; failures return to task profile, label normalization, or evaluator revision.
NPU Optimization	Try runtime settings, `torch_npu` operator substitutions, or a code-level optimized path, then compare baseline and optimized runs.	Optimization record with baseline/perf metrics, speedup, optimization kind, runtime settings, and optimized-path marker.	The faster path must preserve behavior and expose activation evidence; unclear gains or missing records return to optimizer revision.
Application Comparison	Execute NPU baseline/perf scenarios on real weights, collect the matched CUDA reference baseline from the remote host, and summarize the comparison.	NPU/CUDA records with scenario command, quality, latency/throughput, package/device metadata, and optimized-path inheritance evidence.	Acceptance requires aligned task quality, non-degraded NPU self-speedup, positive comparison fields, and complete runtime metadata; broken evidence returns to evaluator, runtime, or remote-result recovery.

Table 3: Hardware profile used in the NPU/CUDA reference comparison. For the NPU target, we report the software-visible topology observed in the empirical runtime records used by this study.

Platform	Public label	Basic data used for interpretation	Role in this paper
A3 NPU server	NPU target path	Software-visible topology exposes two independent NPU dies with separate memory pools.	Target backend. The paper reports dual-die data-parallel aggregate throughput, not shared memory or lower single-request latency.
Remote CUDA reference	CUDA reference path	Matched-scenario CUDA baseline collected remotely under the same workload contract.	Reference backend. It is a remotely executed matched-scenario baseline, not task-quality ground truth.

environment choices, dry-run records, evaluator outputs, quality metrics, optimization notes, runtime settings, failure reasons, and recovered CUDA records. The context gives later agents the information needed to avoid repeating earlier mistakes and to test whether a new claim is compatible with the evidence already collected.

Rich context changes the agent problem. The Functional Validation runner does not merely select a generic metric; it uses adaptation records and model-family metadata to choose a task profile that can expose semantic failures. The optimizer does not merely search for a faster run; it sees which evaluator and output type must remain stable. The Application Comparison agent receives the strongest context because it must test optimized-path inheritance, task-quality preservation, and CUDA scenario matching at the same time. Returning a model to an earlier stage is therefore not a generic failure. It is a context-specific decision that points to the stage whose evidence no longer supports the migration claim.

Table 4: Rich execution context consumed and appended by each migration stage. Later agents use the accumulated records to decide what to revise, optimize, compare, or return to an earlier stage.

Stage	Context consumed	Context appended
Discovery	Public model identity, source URL, model family hints, dependency clues, license and access signals	Registry record, source provenance, model-family label, candidate metadata
Adaptation	Registry metadata, source layout, dependency clues, target backend requirements	Isolated workspace, environment record, dry-run log, device-path evidence, revision notes
Functional Validation	Adaptation workspace, dry-run behavior, output schema, model family and candidate task profile	Dataset and evaluator choice, task outputs, quality metrics, trace records, evaluator failures
NPU Optimization	Functional metrics, baseline artifacts, runtime profile, task output type, unsuccessful optimization attempts	Baseline/perf artifacts, optimization kind, optimized-path record, wall-clock timing, preservation evidence
Application Comparison	All prior context, optimized-path record, task profile, measurement protocol, NPU runtime metadata	NPU baseline/perf records on real weights, recovered CUDA baseline, matched quality and throughput comparison, final validation status

3.3 Execution Evidence Standard

The rich context guides agent decisions; the evidence standard decides acceptance. The core rule is that a portability result is trustworthy only when three views agree. The first view is the recorded output: what the previous stage claims to have produced, such as a targeted optimization or a runtime-only setting. The second view is the runtime record: whether the optimized path was active, and which package versions and device settings were used. The third view is the measurement protocol: whether baseline, perf, and CUDA runs share the same scenario, warmup policy, workload definition, quality metric, and latency scope. A strong result includes all three views. Runtime activation supports optimization claims. Quality preservation supports performance claims. Matched scenario metadata supports hardware-comparison claims.

The standard is checked from Adaptation through Application Comparison. Adaptation asks whether the model can execute on Ascend under its own environment and produce a complete dry-run log. Functional Validation asks whether the adapted model produces task-level metrics under the intended evaluator. Different output types use different metrics; the collection includes generated text, class labels, question answers, embeddings, transcriptions, relevance scores, image outputs, and detection boxes, drawing on task families such as WikiText, MMLU, PubMedQA, LibriSpeech, ScienceQA, SQuAD, CNN/DailyMail, COCO, ImageNet, and GLUE [Merity et al., 2017, Hendrycks et al., 2021, Jin et al., 2019, Panayotov et al., 2015, Lu et al., 2022, Rajpurkar et al., 2016, 2018, Hermann et al., 2015, Lin et al., 2014, Deng et al., 2009, Wang et al., 2019].

NPU Optimization then asks whether a faster target path preserves the same task behavior. Optimization records must identify the measurement protocol, baseline/perf records, wall-clock timing source, output type, workload definition, and optimization kind. Runtime-only optimizations are recorded under their own label, keeping them distinct from code-level changes. If the optimizer creates a code-level optimized path, downstream stages must prove that this path was active in the executed application scenario.

Application Comparison is the final automated check. It runs NPU baseline, NPU perf, and CUDA baseline scenarios on real weights and task data. The CUDA-reference side is collected by the agent instead of being entered manually. After local NPU evidence passes, the Application Comparison agent synchronizes the adaptation to a remote CUDA reference host through SSH. It then executes the CUDA baseline with the matching scenario command, retrieves the resulting records, and runs the completion check. A completed Application Comparison result must contain four kinds of evidence. First, the local NPU self-speedup must be non-degraded. Second, the NPU/CUDA comparison fields must be positive. Third, the task-quality metrics must match the scenario. Fourth, runtime records must describe the executed scenario. These runtime records include scenario commands, package versions, Python executable metadata, warmup counts, task-queue settings, dataset profile, and optimized-path inheritance evidence. The CUDA-reference comparison uses dual-die data-parallel throughput measurements for each NPU/CUDA-validated model. When the evidence breaks, the system returns the item for revision. Examples include quality collapse, NPU perf regression, missing CUDA records, missing measurement fields, NaN or infinite values, SSH or remote bootstrap failure, and optimized paths that do not appear in the executed application scenario [Li et al., 2019, Henderson et al., 2018, Dror et al., 2018, Bouthillier et al., 2021, Kapoor and Narayanan, 2023, Mattson et al., 2020, Reddi et al., 2020].

Table 5 records the major harness refinements that produced the current protocol and motivates the downstream checks discussed next.

Table 5: Major evaluation-harness refinements reconstructed from repository history and project notes.

Period	Harness version	Trigger observed	Validation added
2026/02/18–03/01	Adaptation and functional-validation scaffold	candidate scripts and task metrics	functional-validation artifacts became a required consumer of adaptation dry-run outputs
2026/03/14–03/18	NPU-optimization contract hardening	ambiguous speedup and evidence formats	structured NPU-optimization evidence, workload fields, independent baseline/perf artifacts
2026/03/23–03/25	NPU-optimization scaling pass	runtime-only vs code-level optimizations blurred	explicit optimization kind and optimized-path ownership boundaries
2026/04/05	Application-comparison introduction	phase-local speedups needed application-workload validation	real-data NPU baseline/perf plus agent-managed SSH CUDA reference comparison and package-version tracking
2026/04/07	Application-comparison hardening	quality collapse, NaN/Inf, and insufficient local NPU evidence	quality-drift checks and wait-cuda precheck before remote CUDA deferral
2026/04/08	Remote CUDA recovery hardening	SSH, remote cache, and artifact-sync failures	private remote cache, artifact recovery checks, and conflict isolation
2026/04/22–04/28	Topology and single-card reruns	stale topology and configuration drift	canonical reruns, per-model writeback, and stricter application-comparison status updates

3.4 Downstream Checks and Harness Refinement

Each stage agent follows an execution-grounded revision loop: inspect the assigned adaptation and its accumulated context, run the stage evaluation, capture runtime error or validation output, make a scoped revision, and resubmit. The loop resembles reasoning-action and self-feedback patterns in language-agent work [Yao et al., 2023, Shinn et al., 2023, Madaan et al., 2023, Besta et al., 2024]. The difference is that explicit records and stage checks bound the revision space. The adapter revises imports, device placement, model loading, and dry-run behavior. The Functional Validation runner revises dataset mapping, output schema, trace export, and workload-coverage problems. The optimizer tries runtime settings, `torch_npu`[2] operator substitutions, code-level optimized paths, and performance validation. The Application Comparison agent revises task profiles, label normalization, timing scope, optimized-path inheritance, and remote CUDA collection.

The important rule is that revision must preserve the measurement protocol. If a workload lacks evaluator coverage, the stage agent seeks a better task profile before considering stage completion. If a discrete metric collapses to zero, the Application Comparison agent examines evaluator rules before running CUDA comparison. If remote CUDA stalls, the agent checks process liveness, GPU utilization, result timestamps, and cache-lock behavior before deciding whether to wait, resubmit, or defer remote comparison. A model with local NPU quality collapse returns to evaluator or protocol revision before remote comparison.

The evaluation harness becomes stricter as it encounters agent-produced evidence gaps. We use *harness* broadly: task assignment rules, role instructions, validation checks, required evidence fields, runtime metadata, recovery rules, and downstream acceptance checks. The first version of the system assigned adaptation tasks to multiple adapter agents. Early audits found a recurring failure mode: some results satisfied the surface-level acceptance interface while providing too little evidence for the intended completion claim. The system therefore grew through four validation stages after Discovery: each downstream stage tests a claim made by the previous stage under a stricter execution standard.

[2] `torch_npu` is the PyTorch extension package that exposes Ascend NPU devices, operators, and runtime integration to PyTorch programs.

Three design observations guide the current system. First, validation targets are also optimization targets. If a check rewards artifact presence or a local speedup value, an agent can satisfy that check while missing the later runtime path. Second, validation strength comes from stage diversity and context accumulation: compatibility, task quality, optimized-path inheritance, and CUDA comparability require different evidence, and later stages need the earlier evidence to interpret their own failures. Third, generation and validation must be separated. Stage agents generate candidate evidence, independent validation procedures inspect it, and a result is accepted only after validation. Negative outcomes are also recorded. They distinguish legal access, unsupported formats, local evidence failures, and remote-resource unavailability.

3.5 Scaling the Pipeline with Coordinator-Based Execution

The sequential pipeline above is the logical validation order. At the scale of hundreds of models, executing it manually would make assignment, ownership, and recovery hard to track. SLAI-ASCENDBRIDGE therefore uses a coordinator process to execute the same pipeline at scale. The coordinator assigns eligible records, records ownership and stage start time, receives stage reports, invokes completed-stage checks, updates the migration registry through the public write interface, and releases records after completion or rollback.

Implementation note. The released protocol does not depend on a specific agent runtime. In our runs, a coordinator session managed role-specific stage agents, while validation was performed by the same released workflow records and checks. The sequential pipeline in Figure 1 can be executed by any code-agent or developer workflow that preserves the registry updates, accumulated context, and validation checks.

The role split follows the evidence boundary of the pipeline. The model crawler registers candidates. The adapter owns Adaptation records and dry-run evidence. The Functional Validation runner owns task-level measurement records. The NPU Optimizer owns optimized execution paths and their supporting evidence. The Application Comparison stage agent owns NPU baseline/perf and remote CUDA metrics. The coordinator records completion after checks pass. This keeps stage agents from approving their own outputs and prevents one stage from silently overwriting another stage's evidence, while still letting every stage use the context accumulated before it.

The framework also has a simple persistent-memory layer for cross-task learning. Before a related task starts, a stage agent can read compact role-specific lessons from previous revisions. After the task ends, it can record a short note about reusable failures or fixes. These notes store operational knowledge such as crawler de-duplication rules, isolated environment patterns, evaluator pitfalls, `torch_npu` operator choices, model-family failures, remote CUDA recovery steps, and device-topology repairs. The memory helps later revision attempts start from known failure modes. Acceptance still comes from executable checks and recorded results.

Multiple adapters, Functional Validation runners, or optimizers can work concurrently because assignment records owners and start times. The registry prevents two stage agents from claiming the same in-progress item. The Application Comparison stage is usually serialized because remote CUDA is scarce and its failure modes are expensive. The role split is valuable here because stages provide validation, while coordinator-based execution makes the sequential validation pipeline practical at large scale.

4 Evaluation: Migration Collection, Results, and Runtime Evidence

The evaluation follows a claim ladder. First, we define the accepted migration collection and its released tiers. Second, we report the Application Comparison results that support the hardware-migration claim. Third, we show why stage-local optimization is useful but insufficient without context-aware downstream validation. Finally, we examine how broad model-family coverage stresses the evidence standard.

4.1 Accepted Collection and Released Tiers

The accepted collection contains 500 NPU/CUDA-validated Application Comparison results. A migration enters this collection only after the evidence chain closes: runnable migration, task-level

behavior, optimized-path inheritance, local NPU baseline/perf measurement, and matched CUDA reference comparison. Every result in this section uses the same 500-record collection, so the reported claims are tied to completed comparison evidence rather than workflow inventory.

Collection construction. The candidate pool is built from public model sources with reproducible provenance. The supported entry points are Hugging Face model pages and standalone GitHub project repositories. For hub batches, candidates follow recorded ranking rules based on downloads, trending rank, or creation time, with downloads as the default hot-list order. GitHub entries use public owner/repo metadata. Duplicate model IDs and repository URLs are merged before stage evaluation. When a hub-hosted checkpoint depends on independent project code, the isolated workspace preserves that dependency and runtime validation later checks that the port executes the intended path.

The unit of coverage is the deployable model artifact. In the released 500-record collection, only 42 records are repeated-artifact variants under this definition, so 458 records remain distinct deployment artifacts. Checkpoints that share a backbone but differ in fine-tuning data, task specialization, tokenizer or configuration, quantization or weight format, or repository-level implementation remain separate records when they expose distinct public artifacts. This artifact-level definition is useful for migration because nominally related repositories can exercise different dependency resolution, loading paths, preprocessing logic, dtype behavior, custom code, and optimized-path inheritance. The collection therefore measures robustness across released deployment artifacts, while the tier construction below still preserves family coverage so repeated variants do not dominate smaller public splits. The rows used in this paper come from the 2026/02/08 to 2026/04/09 adaptation campaign. We group the accepted collection by public model-hub and AI-for-Science conventions into six top-level families and 24 fine-grained task families. Figure 2 summarizes the top-level coverage, tier design, and fine-grained spread; Appendix A gives the complete merged table.

Tiered evaluation splits. The released collection is organized into three cumulative tiers. This follows the design principle behind KernelBench, which separates executable GPU-kernel workloads into progressively broader levels [Ouyang et al., 2025]. Our tiers use the same idea for hardware migration: keep a small stable set for rapid iteration, a larger balanced set for primary reporting, and a full challenge set for robustness analysis.

The tier rule has two steps. It first allocates slots across the six top-level families and 24 fine-grained task families so that every tier keeps broad coverage. It then selects records inside each family quota with deterministic result strata over NPU self-speedup, NPU/CUDA throughput ratio, task-quality delta, output slice, dataset, and model identifier. CORE-100 is the fastest public regression tier. STANDARD-300 is the main comparison tier, large enough to cover all fine-grained families while keeping repeated evaluation practical. CHALLENGE-500 is the full NPU/CUDA-validated challenge tier and the main public comparison set.

4.2 Main Application Comparison Results

We first report the final comparison set because this is where a migration claim becomes complete. A migration contributes to the headline results only when real weights, task data, optimized-path inheritance, and matched CUDA comparison agree. This turns agent-produced ports into performance measurements tied to task behavior and runtime evidence.

Application Comparison self-speedup ratio. Among 500 accepted Application Comparison results with valid self-speedup ratios, the mean NPU perf-over-baseline ratio is 1.23x. This supports the claim that the framework can preserve target-side improvements after moving from stage-local optimization to real application workloads. The gain is measured after task-quality and runtime-inheritance checks pass, so the 1.23x mean reflects performance that survives the full Application Comparison boundary.

A3 NPU server versus CUDA reference. On the same 500 NPU/CUDA-validated results, the software-visible dual-die NPU configuration, used as two independent data-parallel workers, reaches a 1.80x mean dual-die/single-die throughput ratio. The same setup reaches a 1.35x mean aggregate-throughput ratio against the CUDA reference baseline and reaches at least CUDA-reference aggregate throughput in 357 of 500 matched scenarios. These results support the practical claim that the

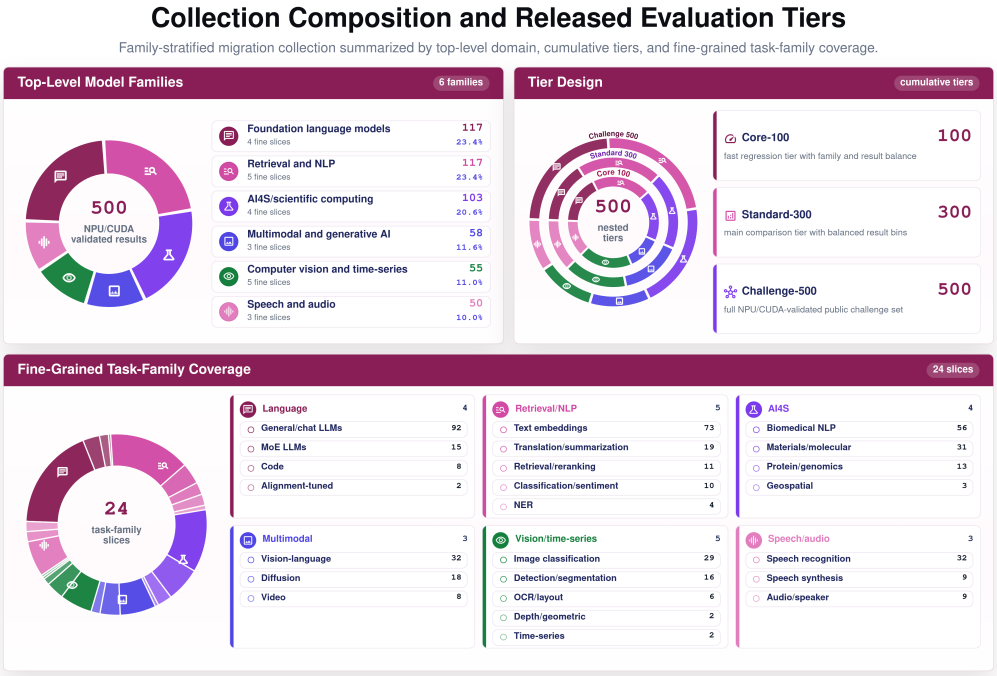


Figure 2: Composition of the 500 NPU/CUDA-validated Application Comparison results and released evaluation tiers. The upper-left panel shows the top-level model-family distribution of the accepted collection. The upper-right panel shows the cumulative CORE-100, STANDARD-300, and CHALLENGE-500 tiers. The bottom panel shows the fine-grained task-family coverage used for tier construction. The tiers preserve family and task coverage, then use result bins for NPU self-speedup, NPU/CUDA throughput ratio, and task-quality delta to keep smaller tiers representative of the full pool.

validated A3 NPU server path is competitive with the CUDA reference for aggregate data-parallel throughput under many real model workloads.

The speed claim is credible because the comparison records also match task quality. All 500 NPU/CUDA-validated results have aligned NPU and CUDA metric names: 398 exactly match the CUDA quality value, 432 are within 10^{-3} absolute quality delta, and 441 are within 10^{-2}. After normalizing metric direction, 51 comparisons favor NPU and 51 favor CUDA by more than numerical equality. This symmetry shows that the throughput result measures comparable application behavior with preserved task quality.

These measurements use data parallelism. Two independent single-die NPU processes run on the two software-visible NPU dies in the A3 server configuration. We sum their completed-request throughput. This directly matches the deployment-oriented claim: when a service can process independent requests in parallel, the aggregate NPU path can convert the second die into measurable throughput. These numbers support aggregate service-throughput interpretation; single-request latency and per-request memory capacity are reported separately when needed.

Table 6 breaks the quality deltas down by metric. Most backend differences are zero or very small under the matched evaluator. This is why the NPU/CUDA result supports the benchmark claim: throughput is reported only after the task metric, dataset, workload definition, and evaluator are matched.

The aggregate mean also reveals why a model-level benchmark is needed. Representative output slices, reported in Appendix Table 11, show that generated text, classification, and question-answering workloads have mean local NPU self-speedup ratios above 1.0x, and their mean dual-die throughput vs CUDA-reference ratios are also above parity. Other slices behave differently. This supports our

Table 6: Matched NPU-vs-CUDA task-quality deltas across all 500 Application Comparison results. Positive and negative directions are normalized per metric when counting which backend is higher; $|\Delta q|$ is the absolute raw metric difference.

Metric	Records	Exact	$\|\Delta q\| \leq 10^{-3}$	NPU higher	CUDA higher	Mean $\|\Delta q\|$
cosine_similarity	127	116/127	125/127	5	6	0.0079
accuracy	118	87/118	87/118	17	14	0.0058
exact_match	93	72/93	72/93	12	9	0.0048
wer	30	20/30	24/30	5	5	0.0006
top1_accuracy	25	25/25	25/25	0	0	0.0000
output_signature_ match_rate	17	17/17	17/17	0	0	0.0000
rougeL	15	10/15	14/15	1	4	0.0003
valid_output_rate	15	15/15	15/15	0	0	0.0000

argument that portability needs model-level evidence, because the bottleneck moves with decoding pattern, preprocessing, batch shape, CPU/NPU boundary behavior, and runtime library path.

Figure 3 turns the result into an actionable validation map. The x-axis asks whether the NPU path improved relative to its own baseline; the y-axis asks whether dual-die throughput vs the CUDA reference is competitive under the same scenario. This two-axis view supports the framework's value because it separates local optimization value from cross-hardware deployment readiness.

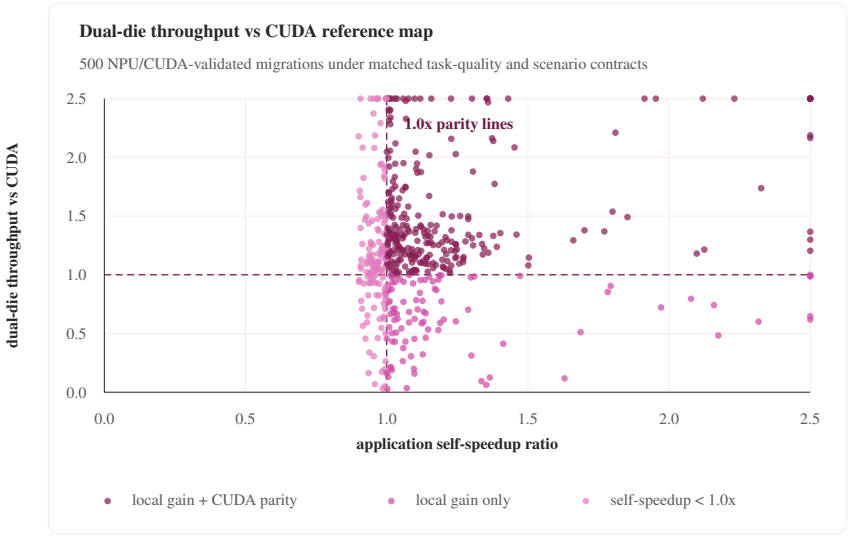


Figure 3: Dual-die throughput versus CUDA-reference diagnostic map. Each point is one NPU/CUDA-validated Application Comparison result. The x-axis checks whether the accepted NPU performance path improves over its own NPU baseline; the y-axis checks whether the A3 NPU server configuration under dual-die data parallelism reaches CUDA-reference aggregate throughput under the same scenario. Parity lines at 1.0x separate local NPU health from cross-hardware competitiveness; point colors mark whether a result is above both parities, above only the local NPU parity, or below local NPU parity.

4.3 Why Downstream Application Comparison Is Necessary

The staged design is necessary because stage-local optimization and application behavior are related but not interchangeable. Within the same 500 accepted records, the joined upstream NPU Optimization evidence has a 2.30x mean stage-local speedup. This shows that the optimization agents often find real target-side opportunities. The later Application Comparison stage asks the harder question with richer context: did the faster path execute under the real workload while preserving task behavior and matching the CUDA scenario?

The transfer audit gives the key evidence. Measured as application self-speedup divided by stage-local optimization speedup, the mean preservation ratio is 87.8%. Of the 59 joined results with NPU Optimization speedup at least 3x, 29 fall below a 1.1x application self-speedup ratio and 13 fall below 1.0x. Conversely, 18 of the 39 Application Comparison results with self-speedup ratio at least 1.5x had NPU Optimization speedup below 1.5x. These asymmetries show that upstream speedup is valuable evidence, but it cannot by itself close a migration claim. Table 7 summarizes the checks.

Table 7: Cross-stage preservation audit for accepted Application Comparison results. Counts are joined by model and compare upstream NPU Optimization speedup with downstream application self-speedup.

Cross-stage check	Count	What it shows
NPU Optimization speedup $\geq$ 3x but application self-speedup ratio $<$ 1.1x	29/59 (49.2%)	large upstream gain needs downstream confirmation
NPU Optimization speedup $\geq$ 3x but application self-speedup ratio $<$ 1.0x	13/59 (22.0%)	downstream execution can expose true regressions despite upstream speedup
Application self-speedup ratio $\geq$ 1.5x with NPU Optimization speedup $<$ 1.5x	18/39 (46.2%)	application gains can come from runtime/data-path changes not captured upstream

4.4 Validation Pressure Across Model Families

The accepted collection is broad enough to stress the evidence standard. The six-family split is more than a coverage summary: it exposes different migration pressures that a one-shot porting prompt would blur together. Foundation models stress generation and routing behavior; retrieval and NLP stress semantic alignment beyond tensor shape; multimodal and generative models stress pipeline completeness; vision, time-series, speech, and audio stress output semantics and preprocessing. Table 8 summarizes how these pressures map to the framework's validation responses.

Table 8: Family-level adaptation pressure in the 500 completed Application Comparison records. Counts are the top-level family counts used in Figure 2. The table shows why one validation protocol must handle several qualitatively different failure modes.

Family in the 500 set	Adaptation pressure	Framework response
Foundation language models (117)	Generation length, tokenizer behavior, cache layout, chat-template choices, and MoE routing can change both latency and answer quality.	Pair generation-aware task checks with runtime records, optimized-path inheritance, and matched CUDA comparison.
Retrieval and NLP (117)	Embedding pooling, sentence-pair inputs, reranking scores, and sequence labels can have correct tensor shapes while losing semantic alignment.	Use output-specific signatures and metric-specific quality checks before a result can enter the comparison set.
AI for Science and scientific computing (103)	Biomedical text, proteins, genomics, materials, and geospatial workloads add domain preprocessing, nonstandard repositories, and output meanings that require domain-aware tasks.	Route models to family-aware tasks. Require domain-appropriate metrics, runtime evidence, and matched CUDA comparison before accepting the migration.
Multimodal and generative AI (58)	Processor chains, image inputs, diffusion stages, and video pipelines can leave only part of the workload on the target backend.	Validate the whole pipeline through task outputs, runtime metadata, and matched application scenarios.
Computer vision and time-series (55)	Classification, detection boxes, masks, feature maps, OCR layout, and forecast horizons require different output semantics.	Apply output-type-specific evaluators so labels, boxes, masks, and horizons remain separately checkable.
Speech and audio (50)	Sampling rate, waveform length, ASR decoding, TTS/vocoder paths, and audio embeddings are sensitive to preprocessing and device placement.	Preserve audio-aware preprocessing and compare task metrics together with runtime and CUDA records.

AI for Science as a stress case. AI for Science is the sharpest example, but it is part of the same validation pattern. These models often look like ordinary PyTorch repositories, while the

portability claim depends on domain-specific data handling and output meaning. Biomedical encoders require task-appropriate text pairs or labels. Protein and genomics models require sequence-level handling rather than generic text generation. Materials and geospatial models often mix scientific preprocessing with model execution. A runnable port can therefore miss the scientific task semantics. SLAI-ASCENDBRIDGE addresses this risk by separating model execution, task-level evaluation, optimized-path inheritance, and matched CUDA comparison. A scientific model is counted as migrated when those checks support the claim.

Task diversity. The Application Comparison summaries cover generated text, classification labels, question answering, embeddings, sentence embeddings, audio transcriptions, relevance scores, detection outputs, and image outputs. The most common recorded datasets include WikiText, MMLU, PubMedQA, LibriSpeech, GSM8K, ScienceQA, SQuAD v2, CIFAR-100, and CNN/DailyMail [Merity et al., 2017, Hendrycks et al., 2021, Jin et al., 2019, Panayotov et al., 2015, Cobbe et al., 2022, Lu et al., 2022, Rajpurkar et al., 2018, Krizhevsky and Hinton, 2009, Hermann et al., 2015]. The fine-grained grouping shows breadth beyond output type. The collection includes general and chat LLMs, MoE LLMs, code models, vision-language models, text-to-image and video generation models, biomedical and clinical NLP, protein and genomics models, materials prediction, geospatial models, retrieval, ASR, TTS, OCR, detection, segmentation, and time-series forecasting. This diversity is part of the validation problem. Different output types need different quality checks. A single similarity rule would make the portability claim weaker, not simpler.

Runtime evidence cases. The case studies show how family-level pressure becomes concrete validation evidence. Each case starts with a migration claim that looked plausible at an upstream boundary. The later Application Comparison stage made the claim measurable under runtime conditions.

Table 9 summarizes representative cases. The Qwen-base biology adaptation is the clearest optimized-path inheritance example. Its NPU Optimization stage was recorded as a fusion-operator path, while the earlier Application Comparison harness ran a different execution path. The NPU perf scenario therefore ran with runtime settings while missing the intended code-level optimization. The application self-speedup ratio was approximately 0.96x, showing that upstream evidence alone did not yet support the application claim. Once Application Comparison consumed the upstream optimization record as part of its required context and checked runtime activation, the application self-speedup ratio recovered to approximately 1.15x.

Table 9: Representative case studies where downstream runtime evidence changed or validated the evaluation standard.

Model family	Validation role	Runtime evidence used in the report
infix-ai/qwen-base-7b-biology	optimized-path inheritance	Application Comparison runtime records made optimized-path loading observable; rerun recovered a valid speedup.
claran/s2orc-biology*	configuration consistency	Single-card NPU reruns and matched CUDA comparison refreshed the Application Comparison evidence.
BAAI/bge-base-en-v1.5	quality-preserving CUDA reference comparison	The application-comparison summary records perfect cosine quality, and the paper reports the matched dual-die throughput vs CUDA reference comparison.
facebook/wav2vec2-base	audio workload	The application-comparison summary records a valid CUDA comparison while preserving cosine-style output consistency under the application-comparison contract.

Two other Application Comparison cases make the same point from different families. One GLiNER case required the evaluator to keep the intended sequence-modeling path on the target device. One MoE case required the application task to exercise routing and aggregation behavior that was not visible in the earlier optimization workload. Together, these cases show why SLAI-ASCENDBRIDGE

evaluates the control loop together with the final numbers. A valid migration record should make the runtime path, task behavior, comparison scenario, and upstream context observable.

5 Limitations

The main limitation is scope. SLAI-ASCENDBRIDGE evaluates validated PyTorch-to-Ascend migration records under the deployment settings covered by the current collection. The three released tiers are stable comparison sets, not new claims beyond the 500 completed records. Their construction keeps family coverage and result distribution visible through the released manifests and the appendix audit. The NPU/CUDA result should be read as an aggregate data-parallel throughput comparison: the A3 NPU server configuration runs two independent software-visible dies as two workers. Future releases can add more deployment settings, revision loops, and context fields while keeping the same core standard: a migration claim should be accepted only when accumulated runtime evidence supports it.

6 Conclusion

SLAI-ASCENDBRIDGE frames PyTorch-to-Ascend migration as context-rich portability validation based on real executions. Its main contribution is an evaluation standard for complete model-level migrations in which later agents consume the accumulated records from earlier stages before producing new evidence. Adaptation dry-run evidence, Functional Validation behavior, optimized-path inheritance, runtime records, and matched CUDA comparison must agree before a portability claim is accepted. The 500-model Application Comparison collection shows that the standard can be applied at scale. The released CORE-100, STANDARD-300, and CHALLENGE-500 tiers make it reusable for external systems.

A Collection Composition Details

Table 10 merges the top-level family coverage, released tier inventory, and fine-grained task-family grouping summarized in Figure 2. The paragraph below gives the exact tier-construction rule. Table 12 audits the coverage and result-balance rule used to construct the released tiers.

Table 10: Merged collection-composition details: top-level model families, cumulative public tiers, and fine-grained task-family slices.

Family or tier	Models	Share	Group	Representative slices or role
Top-level model-family coverage				
Foundation language models	117	23.4%	4 fine slices	General and chat LLMs, MoE LLMs, Code generation and understanding, Alignment and preference-tuned LLMs
Retrieval and NLP	117	23.4%	5 fine slices	Text embeddings and representation, Retrieval engines and reranking, Translation and summarization, Sequence classification and sentiment
AI for Science and scientific computing	103	20.6%	4 fine slices	Biomedical and clinical NLP, Protein and genomics, Materials and molecular prediction, Geospatial and earth science
Multimodal and generative AI	58	11.6%	3 fine slices	Vision-language models, Text-to-image and diffusion, Video generation and understanding
Computer vision and time-series	55	11.0%	5 fine slices	Image classification and feature extraction, Object detection and segmentation, Depth estimation and geometric vision, OCR and document layout
Speech and audio	50	10.0%	3 fine slices	Speech recognition and transcription, Speech synthesis and vocoders, Audio classification and speaker recognition
Released cumulative tiers				

Continued on next page

Table 10: Merged collection-composition details (continued).

Family or tier	Models	Share	Group	Representative slices or role
CORE-100	100	–	24 fine families	Fast regression tier; dominant output slices are class labels, generated text, and CLS embeddings
STANDARD-300	300	–	24 fine families	Main comparison tier; dominant output slices are class labels, generated text, and embeddings
CHALLENGE-500	500	–	24 fine families	Full NPU/CUDA-validated challenge tier; dominant output slices are class labels, generated text, and QA answers
Fine-grained task-family coverage				
General and chat LLMs	92	18.4%	fine family	Llama, Qwen, Mistral
Text embeddings and representation	73	14.6%	fine family	BGE, E5, Nomic Embed
Biomedical and clinical NLP	56	11.2%	fine family	PubMedBERT, ClinicalBERT
Vision-language models	32	6.4%	fine family	Qwen2-VL, LLaVA, Florence
Speech recognition and transcription	32	6.4%	fine family	Whisper, Wav2Vec2
Materials and molecular prediction	31	6.2%	fine family	MoLFormer, MatSciBERT
Image classification and feature extraction	29	5.8%	fine family	DINOv2, Swin, SigLIP
Translation and summarization	19	3.8%	fine family	T5, BART
Text-to-image and diffusion	18	3.6%	fine family	FLUX, Stable Diffusion
Object detection and segmentation	16	3.2%	fine family	YOLO, Grounding DINO, SegFormer
MoE LLMs	15	3.0%	fine family	DeepSeek, Qwen-MoE, Arctic
Protein and genomics	13	2.6%	fine family	ESM, nucleotide models
Retrieval engines and reranking	11	2.2%	fine family	BGE-reranker, ColBERT
Sequence classification and sentiment	10	2.0%	fine family	FinBERT, RoBERTa classifiers
Speech synthesis and vocoders	9	1.8%	fine family	TTS, XTTS, BigVGAN
Audio classification and speaker recognition	9	1.8%	fine family	WavLM, ECAPA-TDNN
Video generation and understanding	8	1.6%	fine family	Wan, LTX
Code generation and understanding	8	1.6%	fine family	StarCoder, Qwen-Coder
OCR and document layout	6	1.2%	fine family	TrOCR, OCR encoders
Named entity recognition	4	0.8%	fine family	BERT-NER, GLiNER
Geospatial and earth science	3	0.6%	fine family	SatVision, geospatial encoders
Depth estimation and geometric vision	2	0.4%	fine family	ZoeDepth
Alignment and preference-tuned LLMs	2	0.4%	fine family	DPO/RLHF-tuned LLMs
Time-series forecasting	2	0.4%	fine family	Chronos, Granite-TTM

Table 11: Task-shape dependence of accepted Application Comparison results. The self-speedup ratio is local NPU perf-over-baseline; dual-die throughput vs CUDA reference is the aggregate-throughput ratio.

Output slice	Records	Mean self-speedup	Mean dual-die/CUDA	$\geq$ CUDA	Reading
generated text	74	1.086x	1.302x	59/74	token-generation throughput is mixed under the dual-die throughput contract
class labels	116	1.122x	1.293x	70/116	classification workloads are mixed under the dual-die throughput contract
qa answers	62	1.216x	1.193x	50/62	quality metric and decoding profile dominate the comparison
transcriptions	29	1.298x	1.842x	21/29	speech workloads are often NPU-competitive
detection boxes	8	1.030x	2.024x	8/8	small structured-vision workloads straddle parity
audio embeddings	9	1.003x	1.159x	5/9	audio embedding workloads are near parity

Tier construction algorithm. The released tier manifests are deterministic. The input is the 500 completed Application Comparison records. Each record provides a model identifier, top-level family, fine-grained task family, output slice, dataset, NPU self-speedup, NPU/CUDA throughput ratio, and absolute task-quality delta. CHALLENGE-500 contains all 500 records. CORE-100 and STANDARD-300 use fixed top-level family quotas:

Tier	Foundation	Retrieval/NLP	AI4S	Multimodal	Vision/time-series	Speech/audio
CORE-100	17	21	18	12	19	13
STANDARD-300	46	64	58	46	47	39

Within each top-level family quota, the algorithm first keeps any records already chosen by the smaller cumulative tier. It then selects at least one record from each available fine-grained task family, as long as the quota has space. The remaining slots are filled greedily. For each candidate record, the algorithm asks how close the selected subset would be to the full family pool after adding that candidate. Closeness is measured over five distributions: fine-grained task family, NPU self-speedup, NPU/CUDA throughput ratio, task-quality delta, and output slice. The result dimensions use fixed bins: NPU self-speedup is grouped at 1.0x, 1.1x, 1.5x, and 2.0x; NPU/CUDA throughput ratio is grouped at 0.5x, 1.0x, 1.5x, and 2.0x; task-quality delta is grouped into exact or near-exact match, below 10^{-3}, below 10^{-2}, and larger differences. The selected candidate is the one that minimizes the largest distribution gap, then the mean distribution gap. Remaining ties are broken by fine-grained task family, output slice, dataset, and model identifier. This rule makes the tiers cumulative, reproducible, and balanced across both task coverage and observed result shape.

Table 12: Tier-balance audit for the released cumulative splits. The construction first preserves family and task coverage, then balances NPU self-speedup, NPU/CUDA throughput ratio, and task-quality delta inside those coverage quotas. Max gap is the largest distribution difference from CHALLENGE-500 across the three result dimensions.

Tier	models	task-family coverage	balanced dimensions	Max gap
Core-100	100	all 24 task families	NPU self-speedup, NPU/CUDA throughput ratio, and task-quality delta	3.4%
Standard-300	300	all 24 task families	NPU self-speedup, NPU/CUDA throughput ratio, and task-quality delta	2.8%
Challenge-500	500	all 24 task families	full released pool; reference distribution for the smaller tiers	0.0%

References

Anthropic. Effective harnesses for long-running agents. `https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents`, 2025. Engineering report.

Anthropic. Demystifying evaluations for ai agents. `https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents`, 2026. Engineering report.

Ascend. Ascend extension for pytorch. `https://github.com/Ascend/pytorch`, 2026.

Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, Michal Podstawski, Lukas Gianinazzi, Joanna Gajda, Tomasz Lehmann, Hubert Niewiadomski, Piotr Nyczyk, and Torsten Hoefler. Graph of thoughts: Solving elaborate problems with large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 17682–17690, 2024.

Xavier Bouthillier, Pierre Delaunay, Mirko Bronzi, Assya Trofimov, Brennan Nichyporuk, Justin Szeto, Nazanin Mohammadi Sepahvand, Edward Raff, Kanika Madan, Vikram Voleti, et al. Accounting for variance in machine learning benchmarks. In *Proceedings of Machine Learning and Systems*, volume 3, pages 747–769, 2021.

Xinzi Cao, Jianyang Zhai, Pengfei Li, Zhiheng Hu, Cen Yan, Bingxu Mu, Guanghuan Fang, Bin She, Jiayu Li, Yihan Su, Dongyang Tao, Xiansong Huang, Fan Xu, Feidiao Yang, Yao Lu, Chang-Dong Wang, Yutong Lu, Weicheng Xue, Bin Zhou, and Yonghong Tian. Ascendkernelgen: A systematic study of llm-based kernel generation for neural processing units, 2026.

Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, et al. TVM: An automated end-to-end optimizing compiler for deep learning. In *13th USENIX Symposium on Operating Systems Design and Implementation*, pages 578–594, 2018.

Yu-Hsin Chen, Tushar Krishna, Joel S. Emer, and Vivienne Sze. Eyeriss: A spatial architecture for energy-efficient dataflow for convolutional neural networks. In *Proceedings of the ACM/IEEE Annual International Symposium on Computer Architecture*, pages 367–379, 2016.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems. In *International Conference on Learning Representations*, 2022.

Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. ImageNet: A large-scale hierarchical image database. In *IEEE Conference on Computer Vision and Pattern Recognition*, pages 248–255, 2009.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 4171–4186, 2019.

David L. Donoho. An invitation to reproducible computational research. *Biostatistics*, 11(3):385–388, 2010.

Rotem Dror, Gili Baumer, Segev Shlomov, and Roi Reichart. The hitchhiker's guide to testing statistical significance in natural language processing. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics*, pages 1383–1392, 2018.

Jinye Du, Quan Yuan, Zuyao Zhang, Yanzhi Yi, Jiahui Hu, Wangyi Chen, Yiyang Zhu, Qishui Zheng, Wenxiang Zou, Xiangyu Chang, Zuohe Zheng, Zichun Ye, et al. Akg kernel agent: A multi-agent framework for cross-platform kernel synthesis, 2025.

Timnit Gebru, Jamie Morgenstern, Briana Vecchione, Jennifer Wortman Vaughan, Hanna Wallach, Hal Daumé III, and Kate Crawford. Datasheets for datasets. *Communications of the ACM*, 64(12): 86–92, 2021.

Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. Deep reinforcement learning that matters. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, 2018.

Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *International Conference on Learning Representations*, 2021.

Karl Moritz Hermann, Tomas Kocisky, Edward Grefenstette, Lasse Espeholt, Will Kay, Mustafa Suleyman, and Phil Blunsom. Teaching machines to read and comprehend. In *Advances in Neural Information Processing Systems*, volume 28, 2015.

Jaber Jaber and Osama Jaber. Autokernel: Autonomous gpu kernel optimization via iterative agent-driven search, 2026.

Zhihao Jia, Oded Padon, James Thomas, Todd Warszawski, Matei Zaharia, and Alex Aiken. TASO: Optimizing deep learning computation with automatic generation of graph substitutions. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles*, pages 47–62, 2019.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues? In *International Conference on Learning Representations*, 2024.

Qiao Jin, Bhuwan Dhingra, Zhengping Liu, William Cohen, and Xinghua Lu. PubMedQA: A dataset for biomedical research question answering. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing and the International Joint Conference on Natural Language Processing*, pages 2567–2577, 2019.

Norman P. Jouppi, Cliff Young, Nishant Patil, David Patterson, Gaurav Agrawal, Raminder Bajwa, Sarah Bates, Suresh Bhatia, Nan Boden, Al Borchers, et al. In-datacenter performance analysis of a tensor processing unit. In *Proceedings of the ACM/IEEE Annual International Symposium on Computer Architecture*, pages 1–12, 2017.

Sayash Kapoor and Arvind Narayanan. Leakage and the reproducibility crisis in machine-learning-based science. *Patterns*, 4(9):100804, 2023.

Alex Krizhevsky and Geoffrey Hinton. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009.

Chris Lattner, Mehdi Amini, Uday Bondhugula, Albert Cohen, Andy Davis, Jacques Pienaar, River Riddle, Tatiana Shpeisman, Nicolas Vasilache, and Oleksandr Zinenko. MLIR: Scaling compiler infrastructure for domain specific computation. In *IEEE/ACM International Symposium on Code Generation and Optimization*, pages 2–14, 2021.

Cheng Li, Abdul Dakkak, Jinjun Xiong, and Wen-mei Hwu. Challenges and pitfalls of machine learning evaluation and benchmarking, 2019.

Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C. Lawrence Zitnick. Microsoft COCO: Common objects in context. In *European Conference on Computer Vision*, pages 740–755, 2014.

Pan Lu, Swaroop Mishra, Tanglin Xia, Liang Qiu, Kai-Wei Chang, Song-Chun Zhu, Oyvind Tafjord, Peter Clark, and Ashwin Kalyan. Learn to explain: Multimodal reasoning via thought chains for science question answering. In *Advances in Neural Information Processing Systems*, volume 35, pages 2507–2521, 2022.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

Peter Mattson, Christine Cheng, Cody Coleman, Greg Diamos, Paulius Micikevicius, David Patterson, Hanlin Tang, Gu-Yeon Wei, Peter Bailis, Victor Bittorf, David Brooks, Dehao Chen, Debojyoti Dutta, Udit Gupta, Kim Hazelwood, Andrew Hock, Xinyuan Huang, Atsushi Ike, Bill Jia, Daniel Kang, David Kanter, Naveen Kumar, Jeffery Liao, Guokai Ma, Deepak Narayanan, Tayo Oguntebi, Gennady Pekhimenko, Lillian Pentecost, Vijay Janapa Reddi, Taylor Robie, Tom St John, Carole-Jean Wu, Lingjie Xu, Cliff Young, and Matei Zaharia. Mlperf training benchmark. In *Proceedings of Machine Learning and Systems*, 2020.

Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. In *International Conference on Learning Representations*, 2017.

Margaret Mitchell, Simone Wu, Andrew Zaldivar, Parker Barnes, Lucy Vasserman, Ben Hutchinson, Elena Spitzer, Inioluwa Deborah Raji, and Timnit Gebru. Model cards for model reporting. In *Proceedings of the Conference on Fairness, Accountability, and Transparency*, pages 220–229, 2019.

Anne Ouyang, Simon Guo, Simran Arora, Alex L. Zhang, William Hu, Christopher Re, and Azalia Mirhoseini. Kernelbench: Can llms write efficient gpu kernels? In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 47356–47415. PMLR, 2025. URL `https://proceedings.mlr.press/v267/ouyang25a.html`.

Vassil Panayotov, Guoguo Chen, Daniel Povey, and Sanjeev Khudanpur. Librispeech: An asr corpus based on public domain audio books. In *IEEE International Conference on Acoustics, Speech and Signal Processing*, pages 5206–5210, 2015.

Joelle Pineau, Philippe Vincent-Lamarre, Koustuv Sinha, Vincent Lariviere, Alina Beygelzimer, Florence d'Alche Buc, Emily Fox, and Hugo Larochelle. Improving reproducibility in machine learning research. *Journal of Machine Learning Research*, 22(164):1–20, 2021.

Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision. In *International Conference on Machine Learning*, pages 8748–8763, 2021.

Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. Robust speech recognition via large-scale weak supervision. In *International Conference on Machine Learning*, pages 28492–28518, 2023.

Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140):1–67, 2020.

Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. SQuAD: 100,000+ questions for machine comprehension of text. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing*, pages 2383–2392, 2016.

Pranav Rajpurkar, Robin Jia, and Percy Liang. Know what you don't know: Unanswerable questions for SQuAD. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics*, pages 784–789, 2018.

Vijay Janapa Reddi, Christine Cheng, David Kanter, Peter Mattson, Guenther Schmuelling, Carole-Jean Wu, Brian Anderson, Maximilien Breughe, Mark Charlebois, William Chou, Ramesh Chukka, Cody Coleman, Sam Davis, Pan Deng, Greg Diamos, Jared Duke, Dave Fick, J. Scott Gardner, Itay Hubara, Sachin Idgunji, Thomas B. Jablin, Jeff Jiao, Tom St. John, Pankaj Kanwar, David Lee, Jeffery Liao, Anton Lokhmotov, Francisco Massa, Peng Meng, Paulius Micikevicius, Colin Osborne, Gennady Pekhimenko, Arun Tejusve Rajan, Dilip Sequeira, Ashish Sirasao, Fei Sun, Hanlin Tang, Michael Thomson, Frank Wei, Ephrem Wu, Lingjie Xu, Koichi Yamada, Bing Yu, George Yuan, Aaron Zhong, Peizhao Zhang, and Yuchen Zhou. Mlperf inference benchmark. In *Proceedings of the ACM/IEEE Annual International Symposium on Computer Architecture*, 2020.

Albert Reuther, Peter Michaleas, Michael Jones, Vijay Gadepally, Siddharth Samsi, and Jeremy Kepner. Ai accelerator survey and trends. In *IEEE High Performance Extreme Computing Conference*, pages 1–9, 2020.

Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 10684–10695, 2022.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, 2023.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, 2023.

Zachary S. Siegel, Sayash Kapoor, Nitya Nadgir, Benedikt Stroebl, and Arvind Narayanan. CORE-Bench: Fostering the credibility of published research through a computational reproducibility agent benchmark. *Transactions on Machine Learning Research*, 2025.

Atharv Sonwane, Eng-Shen Tu, Wei-Chung Lu, Claas Beger, Carter Larsen, Debjit Dhar, Simon Alford, Rachel Chen, Ronit Pattanayak, Tuan Anh Dang, Guohao Chen, Gloria Geng, Kevin Ellis, and Saikat Dutta. Omnicode: A benchmark for evaluating software engineering agents, 2026.

Xianpeng (Simon) Sun, Haonan Sun, Tian Yu, Sheng Ma, Qincheng Zhang, Lifei Rao, and Chen Tian. A time-consistent benchmark for repository-level software engineering evaluation, 2026.

Vivienne Sze, Yu-Hsin Chen, Tien-Ju Yang, and Joel S. Emer. Efficient processing of deep neural networks: A tutorial and survey. *Proceedings of the IEEE*, 105(12):2295–2329, 2017.

Philippe Tillet, H. T. Kung, and David Cox. Triton: An intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, pages 10–19, 2019.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

Konstantinos Vergopoulos, Mark Niklas Müller, and Martin Vechev. Automated benchmark generation for repository-level coding tasks. In *International Conference on Machine Learning*, 2025.

vLLM Ascend Contributors. vllm ascend documentation. https://docs.vllm.ai/projects/ascend/en/latest/, 2026.

Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. GLUE: A multi-task benchmark and analysis platform for natural language understanding. In *International Conference on Learning Representations*, 2019.

Zhongzhen Wen, Yinghui Zhang, Zhong Li, Zhongxin Liu, Linna Xie, and Tian Zhang. Multikernelbench: A multi-platform benchmark for kernel generation, 2025.

Zhongzhen Wen, Shudi Shao, Zhong Li, Yu Ge, Tongtong Xu, Yuanyi Lin, and Tian Zhang. Ascendcraft: Automatic ascend npu kernel generation via dsl-guided transcompilation, 2026.

Greg Wilson, D. A. Aruliah, C. Titus Brown, Neil P. Chue Hong, Matt Davis, Richard T. Guy, Steven H. D. Haddock, Katy Huff, Ian M. Mitchell, Mark Plumbley, Ben Waugh, Ethan P. White, and Paul Wilson. Best practices for scientific computing. *PLOS Biology*, 12(1):e1001745, 2014.

Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Demystifying llm-based software engineering agents. *Proceedings of the ACM on Software Engineering*, 2(FSE):801–824, 2025. doi: 10.1145/3715754.

John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems*, volume 37, 2024. doi: 10.52202/079017-1601.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.

Yang Yu, Peiyu Zang, Chi Hsu Tsai, Haiming Wu, Yixin Shen, Jialing Zhang, Haoyu Wang, Zhiyou Xiao, Jingze Shi, Yuyu Luo, Wentao Zhang, Chunlei Men, et al. Towards automated kernel generation in the era of llms, 2026.

Dewu Zheng, Yanlin Wang, Ensheng Shi, Ruikai Zhang, Yuchi Ma, Hongyu Zhang, and Zibin Zheng. HumanEvo: An evolution-aware benchmark for more realistic evaluation of repository-level code generation. In *IEEE/ACM International Conference on Software Engineering*, 2025.

Jiace Zhu, Wentao Chen, Qi Fan, Zhixing Ren, Junying Wu, Xing Zhe Chai, Chotiwit Rungrueangwutthinon, Yehan Ma, and An Zou. Cudabench: Benchmarking llms for text-to-cuda generation, 2026.

Xingze Zou, Jing Wang, Yuhua Zheng, Xueyi Chen, Haolei Bai, Lingcheng Kong, Syed A. R. Abu-Bakar, Zhaode Wang, Chengfei Lv, Haoji Hu, Huan Wang, et al. Mobilekernelbench: Can llms write efficient kernels for mobile devices?, 2026.